
linux-server-assignment

**Sou Chanrojame, Orn Pheakdey, Long Neron,
Le Sreyma, Then Sivthean**

Apr 08, 2026

CONTENTS:

1	File Server	3
1.1	Introduction	3
1.2	Implementation	3
1.3	Usage	4
2	Proxy Server	5
2.1	Introduction	5
2.2	Implementation	5
2.3	Usage	7
3	DNS Server	9
3.1	Introduction	9
3.2	Implementation	9
3.3	Usage	11
4	DHCP Server	13
4.1	Introduction	13
4.2	Implementation	13
4.3	Usage	15
5	VPN Server	17
5.1	Introduction	17
5.2	Implementation	17
5.3	Usage	19
6	Terminal Server	21
6.1	Introduction	21
6.2	Implementation	21
6.3	Usage	22
7	Web Server	23
7.1	Introduction	23
7.2	Implementation	23
7.3	Usage	23
8	Mail Server	25
8.1	Introduction	25
8.2	Implementation	25
8.3	Usage	26
9	Database Server	29

9.1	Introduction	29
9.2	Implementation	29
9.3	Usage	30
10	Backup Server	35
10.1	Introduction	35
10.2	Implementation	35
10.3	Usage	36
11	Load Balancing	37
11.1	Introduction	37
11.2	Implementation	37
11.3	Usage	37
12	Failover Cluster	39
12.1	Introduction	39
12.2	Implementation	39
12.3	Usage	40
13	FTP Server	43
13.1	Introduction	43
13.2	Implementation	43
13.3	Usage	44
14	Docker	45
14.1	Introduction	45
14.2	Implementation	45
14.3	Usage	46
15	Domain Controller	47
15.1	Introduction	47
15.2	Implementation	47
15.3	Usage	47
16	Compose Yaml	51

This is linux-server-assignment which include 15 labs.

FILE SERVER

1.1 Introduction

Samba is an open-source software suite that enables file and printer sharing between Linux/Unix systems and Windows clients using the SMB/CIFS protocol. It allows a Linux server to function as a file server, providing centralized storage, shared directories, and access control for multiple users across a network. A file server stores and manages files for clients, enabling users to read, write, and collaborate on data securely, while Samba ensures compatibility with Windows environments and seamless integration into mixed-network infrastructures.

1.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install -y samba samba-common samba-client

mkdir -p /srv/samba/shared
chmod 777 /srv/samba/shared

tee /etc/samba/smb.conf > /dev/null <<EOF
[global]
workgroup = WORKGROUP
server string = CentOS File Server
netbios name = centos-fs
security = user
map to guest = bad user
dns proxy = no

[shared]
path = /srv/samba/shared
browseable = yes
writable = yes
guest ok = yes
read only = no
force create mode = 0775
force directory mode = 0775
EOF

# background/daemon => run in other process
```

(continues on next page)

(continued from previous page)

```
# foreground/interactive => run in the main process
exec smbd --foreground --no-process-group
```

1.3 Usage

Connect to file server instance with ssh.

```
sudo dnf install -y samba-client
smbclient //localhost/shared -N

# +-----+
# | Sample Output |
# +-----+
# jame@Jame-Linux ~/Desktop/coding/linux-server-assignment/docs % smbclient //
↪localhost/shared -N
# Can't load /etc/samba/smb.conf - run testparm to debug it
# Anonymous login successful
# Try "help" to get a list of possible commands.
# smb: \>
```

```
[ec2-user@ip-172-31-68-111 linux-server-assignment]$ smbclient //localhost/shared
Password for [SAMBA\ec2-user]:
Try "help" to get a list of possible commands.
smb: \> dir
.                D            0   Sat Apr  4 09:00:30 2026
..               D            0   Sat Apr  4 09:00:30 2026

      36622316 blocks of size 1024. 33706508 blocks available
smb: \> put README.md
putting file README.md as \README.md (566.4 kb/s) (average 566.4 kb/s)
smb: \> put run.sh
putting file run.sh as \run.sh (5748.5 kb/s) (average 2293.9 kb/s)
smb: \> dir
.                D            0   Sat Apr  4 09:00:55 2026
..               D            0   Sat Apr  4 09:00:55 2026
README.md        A          1160   Sat Apr  4 09:00:50 2026
run.sh           A          5887   Sat Apr  4 09:00:55 2026

      36622316 blocks of size 1024. 33706456 blocks available
smb: \> put LICENSE
putting file LICENSE as \LICENSE (2772.6 kb/s) (average 2567.5 kb/s)
smb: \> put compose.yaml
putting file compose.yaml as \compose.yaml (3140.5 kb/s) (average 2694.9 kb/s)
smb: \> █
```


PROXY SERVER

2.1 Introduction

Squid is a caching and forwarding proxy server that handles web requests on behalf of clients, improving performance and controlling access to the internet. By storing frequently accessed content locally, Squid reduces bandwidth usage and speeds up response times for repeated requests. A proxy server acts as an intermediary between client devices and external servers, providing benefits like content filtering, anonymity, security, and traffic management, making it useful for organizations to optimize and regulate internet access.

2.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install -y squid

tee /etc/squid/squid.conf > /dev/null <<EOF
#
# Recommended minimum configuration:
#
# Example rule allowing access from your local networks.
# Adapt to list your (internal) IP networks from where browsing
# should be allowed

acl localnet src 10.0.0.0/8          # RFC1918 possible internal network
acl localnet src 172.16.0.0/12       # RFC1918 possible internal network
acl localnet src 192.168.0.0/16      # RFC1918 possible internal network
acl localnet src fc00::/7           # RFC 4193 local private network range
acl localnet src fe80::/10          # RFC 4291 link-local (directly plugged) machines

# acl localnet src 172.31.74.0/24 # not needed, because it is included in the above.

acl SSL_ports port 443
acl Safe_ports port 80              # http
acl Safe_ports port 21              # ftp
acl Safe_ports port 443             # https
acl Safe_ports port 70              # gopher
acl Safe_ports port 210             # wais
acl Safe_ports port 1025-65535      # unregistered ports
```

(continues on next page)

(continued from previous page)

```

acl Safe_ports port 280          # http-mgmt
acl Safe_ports port 488          # gss-http
acl Safe_ports port 591          # filemaker
acl Safe_ports port 777          # multiling http
acl CONNECT method CONNECT

acl block_fb dstdomain .facebook.com .fbcdn.net .messenger.com
acl block_instagram dstdomain .instagram.com

#
# Recommended minimum Access Permission configuration:
#

# Only allow cachemgr access from localhost
http_access allow localhost manager
http_access deny manager

# Deny requests to certain unsafe ports
http_access deny !Safe_ports

# Deny CONNECT to other than secure SSL ports
http_access deny CONNECT !SSL_ports

# We strongly recommend the following be uncommented to protect innocent
# web applications running on the proxy server who think the only
# one who can access services on "localhost" is a local user
#http_access deny to_localhost

#
# INSERT YOUR OWN RULE(S) HERE TO ALLOW ACCESS FROM YOUR CLIENTS
#

# Example rule allowing access from your local networks.
# Adapt localnet in the ACL section to list your (internal) IP networks
# from where browsing should be allowed
http_access deny block_fb
http_access deny block_instagram

http_access allow localnet
http_access allow localhost

# And finally deny all other access to this proxy
http_access deny all

# Squid normally listens to port 3128
http_port 3128

# Uncomment the line below to enable disk caching - path format is /cygdrive/<full_
→path to cache folder>, i.e.
#cache_dir aufs /cygdrive/d/squid/cache 3000 16 256

```

(continues on next page)

(continued from previous page)

```
# Leave coredumps in the first cache dir
coredump_dir /var/cache/squid

# Add any of your own refresh_pattern entries above these.
refresh_pattern ^ftp:          1440      20%      10080
refresh_pattern ^gopher:      1440       0%       1440
refresh_pattern -i (/cgi-bin/|\?) 0        0%        0
refresh_pattern .              0         20%      4320

dns_nameservers 8.8.8.8 208.67.222.222

max_filedescriptors 3200
EOF

exec squid --foreground
```

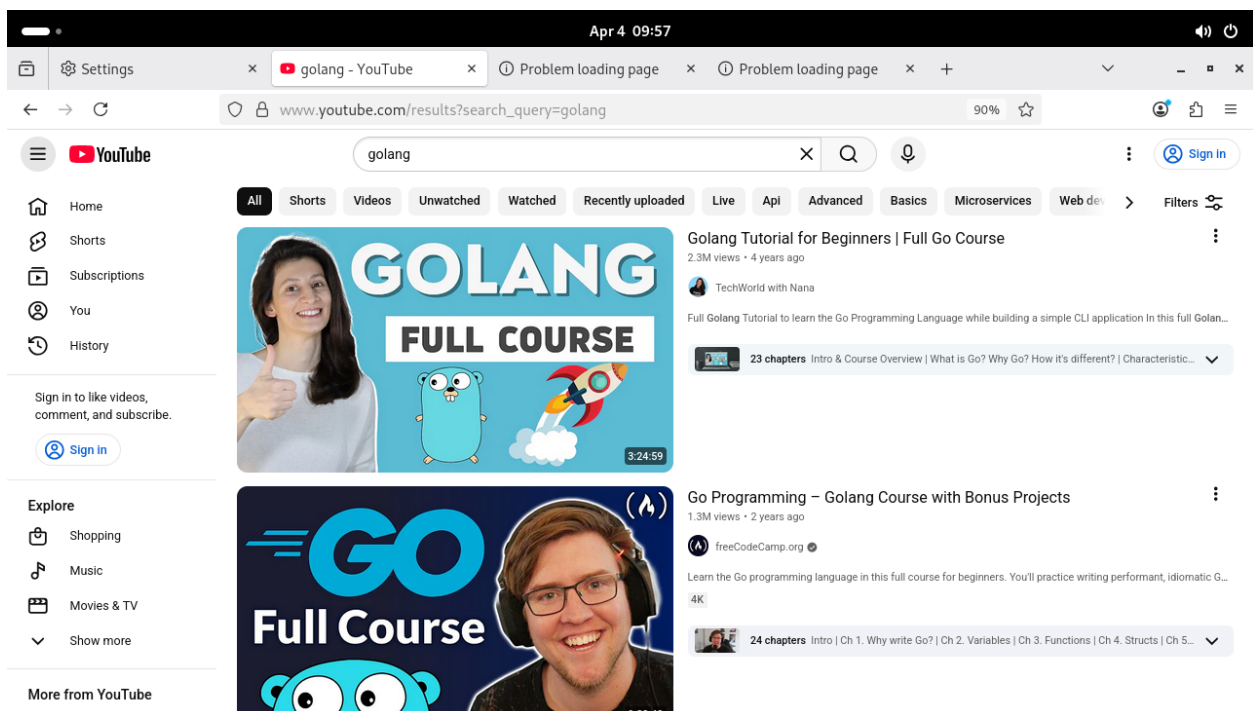
2.3 Usage

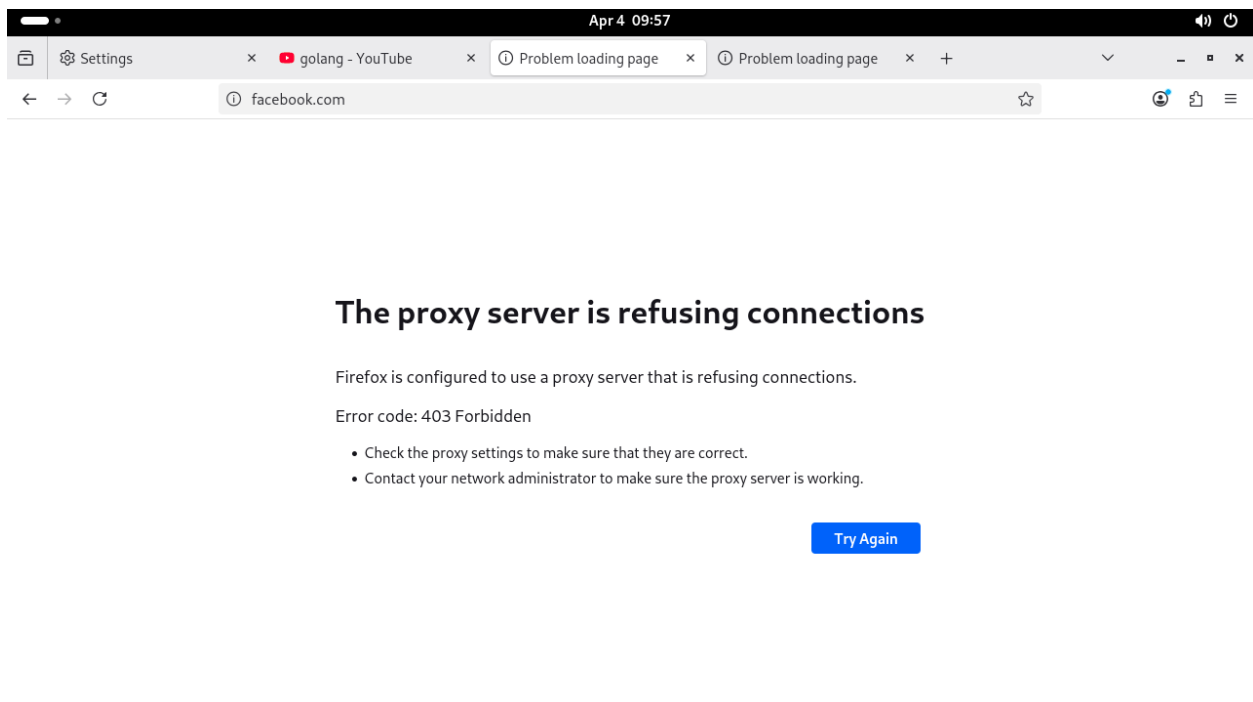
```
ssh -i <keypair> -L 5901:localhost:5901 ec2-user@<address>
```

Use your VNC client to connect to localhost:5901 or 127.0.0.1:5901 with password 'dog@123'

Open firefox and change proxy to localhost with port 3128 (make sure *https* is checked)

- visit youtube.com => allow
- visit facebook.com => blocked





DNS SERVER

3.1 Introduction

BIND is the most widely used software for implementing a Domain Name System (DNS) server, which translates human-readable domain names into IP addresses that computers use to locate each other on a network. BIND allows administrators to configure authoritative DNS zones, manage domain records, and handle recursive queries, enabling reliable name resolution for both internal networks and the internet. A DNS server, in general, is essential for directing network traffic efficiently, supporting web browsing, email delivery, and other services that rely on domain names.

3.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install -y bind bind-utils

mkdir -p /run/named /var/named/dynamic
chown -R named:named /run/named /var/named
chmod 777 /run/named /var/named

# Configure BIND
tee /etc/named.conf > /dev/null <<EOF
options {
    listen-on port 53 { any; };
    listen-on-v6 port 53 { ::1; };
    directory "/var/named";
    dump-file "/var/named/data/cache_dump.db";
    statistics-file "/var/named/data/named_stats.txt";
    memstatistics-file "/var/named/data/named_mem_stats.txt";
    # allow-query { localhost; 192.168.0.0/16; 10.0.0.0/8; };
    allow-query { any; };
    recursion yes;
    dnssec-validation yes;
    managed-keys-directory "/var/named/dynamic";
    pid-file "/run/named/named.pid";
    session-keyfile "/run/named/session.key";
};

logging {
    channel default_debug {
```

(continues on next page)

(continued from previous page)

```
        file "data/named.run";
        severity dynamic;
    };
};

zone "example.internal" IN {
    type master;
    file "example.internal.dns";
};

zone "devspeed.com" IN {
    type master;
    file "devspeed.com.dns";
};

zone "." IN {
    type hint;
    file "named.ca";
};

include "/etc/named.rfc1912.zones";
include "/etc/named.root.key";
EOF

tee /var/named/example.internal.dns > /dev/null <<"EOF"
$TTL 86400
@   IN  SOA ns1.example.internal. admin.example.internal. (
        2026020601 ; Serial
        3600      ; Refresh
        1800      ; Retry
        604800    ; Expire
        86400 )   ; Minimum

; Name servers
@       IN  NS   ns1.example.internal.

; A records
ns1      IN  A   10.0.1.10
server1  IN  A   10.0.1.10
EOF

tee /var/named/devspeed.com.dns > /dev/null <<"EOF"
$TTL 86400
@   IN  SOA ns1.devspeed.com. admin.devspeed.com. (
        2026020601
        3600
        1800
        604800
        86400 )

; Name servers
@       IN  NS   ns1.devspeed.com.
```

(continues on next page)

(continued from previous page)

```
; A records
ns1      IN  A    192.168.1.1
console  IN  A    192.168.1.11
go       IN  A    192.168.1.29
blog     IN  A    192.168.1.30
shop     IN  A    192.168.1.31
support  IN  A    192.168.1.32
mail     IN  A    192.168.1.33
www      IN  A    192.168.1.34
www2     IN  A    192.168.1.100
EOF

exec named -g -u named
```

3.3 Usage

```
dog console.devspeed.com @127.0.0.1 # replace 127.0.0.1 with ip address of dns server
dog go.devspeed.com @127.0.0.1
dog blog.devspeed.com @127.0.0.1
dog shop.devspeed.com @127.0.0.1
dog support.devspeed.com @127.0.0.1
dog mail.devspeed.com @127.0.0.1
dog www.devspeed.com @127.0.0.1
dog www2.devspeed.com @127.0.0.1

# +-----+
# | Sample Output |
# +-----+
# [ 11:17AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog console.devspeed.com @127.0.0.1
# A console.devspeed.com. 1d0h00m00s 192.168.1.11
# [ 11:18AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog go.devspeed.com @127.0.0.1
# A go.devspeed.com. 1d0h00m00s 192.168.1.29
# [ 11:18AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog blog.devspeed.com @127.0.0.1
# A blog.devspeed.com. 1d0h00m00s 192.168.1.30
# [ 11:18AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog shop.devspeed.com @127.0.0.1
# A shop.devspeed.com. 1d0h00m00s 192.168.1.31
# [ 11:19AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog support.devspeed.com @127.0.0.1
# A support.devspeed.com. 1d0h00m00s 192.168.1.32
# [ 11:19AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
```

(continues on next page)

(continued from previous page)

```
# $ dog mail.devspeed.com @127.0.0.1
# A mail.devspeed.com. 1d0h00m00s 192.168.1.33
# [ 11:20AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog www.devspeed.com @127.0.0.1
# A www.devspeed.com. 1d0h00m00s 192.168.1.34
# [ 11:20AM ] [ ]
→jame@Jame-Linux:~/Desktop/coding/linux-server-assignment/src(master) ]
# $ dog www2.devspeed.com @127.0.0.1
# A www2.devspeed.com. 1d0h00m00s 192.168.1.100
```

```
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog console.devspeed.com @44.200.93.80 • *master
A console.devspeed.com. 1d0h00m00s 192.168.1.11
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog go.devspeed.com @44.200.93.80 • *master
A go.devspeed.com. 1d0h00m00s 192.168.1.29
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog blog.devspeed.com @44.200.93.80 • *master
A blog.devspeed.com. 1d0h00m00s 192.168.1.30
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog shop.devspeed.com @44.200.93.80 • *master
A shop.devspeed.com. 1d0h00m00s 192.168.1.31
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog support.devspeed.com @44.200.93.80 • *master
A support.devspeed.com. 1d0h00m00s 192.168.1.32
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog mail.devspeed.com @44.200.93.80 • *master
A mail.devspeed.com. 1d0h00m00s 192.168.1.33
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog www.devspeed.com @44.200.93.80 • *master
A www.devspeed.com. 1d0h00m00s 192.168.1.34
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ dog www2.devspeed.com @44.200.93.80 • *master
A www2.devspeed.com. 1d0h00m00s 192.168.1.100
jame@Jame-Linux ~/Desktop/coding/linux-server-assignment$ • *master
```


DHCP SERVER

4.1 Introduction

A DHCP (Dynamic Host Configuration Protocol) Server is a network service that automatically assigns IP addresses and other critical network configuration parameters (such as subnet masks, default gateways, and DNS server addresses) to devices (clients) when they connect to a network.

4.2 Implementation

4.2.1 Server

```
#!/usr/bin/env bash

# can use dhcpd/dnsmasq/udhcpd

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

# Install DHCP server
dnf install -y dhcp-server iproute

# 1. Get the primary network interface
INTERFACE=$(ip route get 1.1.1.1 | awk '{print $5}')

# 2. Extract Network Information using 'ip'
ADDR_INFO=$(ip -4 addr show $INTERFACE | grep -oP 'inet \K[\d./]+')
CURRENT_IP=$(echo $ADDR_INFO | cut -d/ -f1)
CIDR=$(echo $ADDR_INFO | cut -d/ -f2)

# Broadcast Address
BROADCAST=$(ip -4 addr show $INTERFACE | awk '/brd / {print $4}')

# Network Address (e.g., 172.17.0.0)
NETWORK=$(ip route list dev $INTERFACE | awk '/proto kernel/ {print $1}' | cut -d/ -f1)

# Router (Your current Gateway)
ROUTER=$(ip route show default | awk '/default/ {print $3}')

# Helper: Convert CIDR prefix to Dotted Decimal Netmask
mask_conv() {
```

(continues on next page)

(continued from previous page)

```

local cidr=$1
local mask=""
for ((i=0; i<4; i++)); do
    local part=0
    if [ $cidr -ge 8 ]; then
        part=255
        cidr=$((cidr - 8))
    elif [ $cidr -gt 0 ]; then
        part=$((256 - 2**(8 - cidr)))
        cidr=0
    fi
    mask+="{part}"
    [ $i -lt 3 ] && mask+="."
done
echo $mask
}

NETMASK=$(mask_conv $CIDR)
PREFIX=$(echo $CURRENT_IP | cut -d. -f1-3)

# Configure DHCP
tee /etc/dhcp/dhcpd.conf > /dev/null <<EOF
# DHCP Server Configuration
option domain-name "example.com";
option domain-name-servers 8.8.8.8, 8.8.4.4;

default-lease-time 600;
max-lease-time 7200;
authoritative;

# subnet 192.168.1.0 netmask 255.255.255.0 {
#     range 192.168.1.100 192.168.1.200;
#     option routers 192.168.1.1;
#     option subnet-mask 255.255.255.0;
#     option broadcast-address 192.168.1.255;
# }

# subnet 172.17.0.0 netmask 255.255.0.0 {
#     range 172.17.0.100 172.17.0.200;
#     option routers 172.17.0.1;
#     option subnet-mask 255.255.0.0;
#     option broadcast-address 172.17.255.255;
# }

subnet $NETWORK netmask $NETMASK {
    range ${PREFIX}.100 ${PREFIX}.200;
    option routers $ROUTER;
    option subnet-mask $NETMASK;
    option broadcast-address $BROADCAST;
}
EOF

```

(continues on next page)

(continued from previous page)

```
exec dhcpcd -d
```

4.2.2 Client

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

# Install DHCP client
dnf install -y dhcp-client
```

4.3 Usage

Connect to dhcp server instance with ssh.

```
docker exec -it linux-server-assignment-dhcp_client-1 bash
bash /root/dhcp_client.sh

dhclient -d eth0

# +-----+
# | Sample Output: |
# +-----+
# bash-5.1# dhclient -d eth0
# Internet Systems Consortium DHCP Client 4.4.2b1
# Copyright 2004-2019 Internet Systems Consortium.
# All rights reserved.
# For info, please visit https://www.isc.org/software/dhcp/
#
# RTNETLINK answers: Operation not permitted
# Listening on LPF/eth0/06:c4:ca:0d:c2:96
# Sending on LPF/eth0/06:c4:ca:0d:c2:96
# Sending on Socket/fallback
# DHCPREQUEST for 172.17.0.100 on eth0 to 255.255.255.255 port 67 (xid=0x89771024)
# DHCPACK of 172.17.0.100 from 172.17.0.2 (xid=0x89771024)
# RTNETLINK answers: Operation not permitted
# RTNETLINK answers: Operation not permitted
# System has not been booted with systemd as init system (PID 1). Can't operate.
# Failed to connect to bus: Host is down
# bound to 172.17.0.100 -- renewal in 270 seconds.
```

```
bash-5.1# dhclient -d eth0
Internet Systems Consortium DHCP Client 4.4.2b1
Copyright 2004-2019 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

RTNETLINK answers: Operation not permitted
Listening on LPF/eth0/ee:5c:ea:9f:ba:92
Sending on   LPF/eth0/ee:5c:ea:9f:ba:92
Sending on   Socket/fallback
DHCPREQUEST for 172.18.0.100 on eth0 to 255.255.255.255 port 67 (xid=0x5ba02c57)
DHCPACK of 172.18.0.100 from 172.18.0.4 (xid=0x5ba02c57)
RTNETLINK answers: Operation not permitted
RTNETLINK answers: Operation not permitted
System has not been booted with systemd as init system (PID 1). Can't operate.
Failed to connect to bus: Host is down
bound to 172.18.0.100 -- renewal in 240 seconds.
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
DHCPREQUEST for 172.18.0.100 on eth0 to 172.18.0.4 port 67 (xid=0x5ba02c57)
```

VPN SERVER

5.1 Introduction

A VPN (Virtual Private Network) server acts as a secure, intermediate gateway between a user's device and the internet, establishing an encrypted "tunnel" for all data traffic. It is the cornerstone of privacy-focused browsing and secure remote access.

5.2 Implementation

5.2.1 Server

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install epel-release -y

# Install OpenVPN and Easy-RSA
dnf install -y openvpn easy-rsa iptables

# Remove existing openvpn files
rm -rf /etc/openvpn/*

# Setup Easy-RSA
mkdir -p /etc/openvpn/easy-rsa
cp -r /usr/share/easy-rsa/3/* /etc/openvpn/easy-rsa/
cd /etc/openvpn/easy-rsa

# Initialize PKI
./easyrsa init-pki
# Build CA
./easyrsa build-ca nopass <<< "OpenVPN-CA"
# Generate server request
./easyrsa gen-req server nopass <<< "server"
# Sign server certificate
./easyrsa sign-req server server <<< "yes"
# Generate DH parameters
./easyrsa gen-dh
# Generate TLS auth key
openvpn --genkey secret /etc/openvpn/ta.key
```

(continues on next page)

(continued from previous page)

```
# Copy certificates
cp pki/ca.crt pki/private/server.key pki/issued/server.crt pki/dh.pem /etc/openvpn/

# Configure OpenVPN
tee /etc/openvpn/server.conf > /dev/null <<EOF
port 1194
proto udp
dev tun
ca ca.crt
cert server.crt
key server.key
dh dh.pem
server 10.8.0.0 255.255.255.0
ifconfig-pool-persist ipp.txt
push "redirect-gateway def1 bypass-dhcp"
push "dhcp-option DNS 8.8.8.8"
push "dhcp-option DNS 8.8.4.4"
keepalive 10 120
tls-auth ta.key 0
cipher AES-256-CBC
user nobody
group nobody
persist-key
persist-tun
status openvpn-status.log
verb 3
EOF

# Generate a client certificate
cd /etc/openvpn/easy-rsa
./easyrsa gen-req client1 nopass <<< 'client1'
./easyrsa sign-req client client1 <<< 'yes'

# Copy to clients
mkdir -p /etc/openvpn/clients
cp pki/private/client1.key pki/issued/client1.crt pki/ca.crt /etc/openvpn/ta.key /
  ↪etc/openvpn/clients/

# Make client .ovpn
cd /etc/openvpn/clients/
cat > /etc/openvpn/clients/client1.ovpn <<EOF
client
dev tun
proto udp
# remote YOUR_SERVER_IP 1194
remote vpn_server 1194
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
cipher AES-256-CBC
```

(continues on next page)

(continued from previous page)

```

tls-auth ta.key 1
verb 3
<ca>
$(cat ca.crt)
</ca>
<cert>
$(cat client1.crt)
</cert>
<key>
$(cat client1.key)
</key>
<tls-auth>
$(cat ta.key)
</tls-auth>
EOF

# Enable IP forwarding
# sysctl -w net.ipv4.ip_forward=1
# echo 1 > /proc/sys/net/ipv4/ip_forward
# Masquerade VPN subnet
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
# iptables -A FORWARD -i tun0 -j ACCEPT
# iptables -A FORWARD -o tun0 -j ACCEPT

# Starting OpenVPN
exec openvpn --cd /etc/openvpn --config server.conf

```

5.2.2 Client

```

#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install epel-release -y

# Install OpenVPN and Easy-RSA
dnf install -y openvpn easy-rsa iproute

openvpn --config /etc/openvpn/clients/client1.ovpn --daemon

```

5.3 Usage

Connect to vpn server instance with ssh.

Before running this, make sure the vpn server have already finished stating the server.

```

# run bash in the client
docker exec -it linux-server-assignment-vpn_client-1 bash

# Connect to the vpn server
bash /root/vpn_client.sh

```

(continues on next page)

(continued from previous page)

```
# Check OpenVPN tunnel is up inside the client container
```

```
ip addr show tun0
```

```
# Check the client's routing table – default route should go through tun0
```

```
ip route
```

```
# Check assigned VPN IP (should be in 10.8.0.x range)
```

```
ip addr show tun0 | grep inet
```

```
[ec2-user@ip-172-31-65-0 ~]$ docker exec -it linux-server-assignment-vpn_client-1 bash
[root@9c479891a020 ~]# ip addr show tun0
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.8.0.6 peer 10.8.0.5/32 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::7aa5:43f4:d42f:84a6/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever
[root@9c479891a020 ~]# ip route
0.0.0.0/1 via 10.8.0.5 dev tun0
default via 172.18.0.1 dev eth0
10.8.0.1 via 10.8.0.5 dev tun0
10.8.0.5 dev tun0 proto kernel scope link src 10.8.0.6
128.0.0.0/1 via 10.8.0.5 dev tun0
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.2
172.18.0.3 via 172.18.0.1 dev eth0
[root@9c479891a020 ~]# ip addr show tun0 | grep inet
    inet 10.8.0.6 peer 10.8.0.5/32 scope global tun0
    inet6 fe80::7aa5:43f4:d42f:84a6/64 scope link stable-privacy proto kernel_ll
[root@9c479891a020 ~]#
```


TERMINAL SERVER

6.1 Introduction

TigerVNC is a high-performance, platform-independent implementation of the Virtual Network Computing (VNC) protocol that allows users to remotely access and control graphical desktops on a server. A terminal server provides multiple users with simultaneous access to a centralized server environment, letting them run applications and desktops remotely while all processing happens on the server. Together, TigerVNC enables secure remote graphical sessions, effectively turning a terminal server into an accessible desktop environment for users from anywhere on the network.

6.2 Implementation

```
#!/usr/bin/env bash

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install -y epel-release
dnf config-manager --set-enabled crb
dnf group install "Xfce" -y
dnf install -y tigervnc-server

mkdir -p ~/.vnc
cat << EOF > ~/.vnc/xstartup
#!/bin/sh

unset SESSION_MANAGER
unset DBUS_SESSION_BUS_ADDRESS

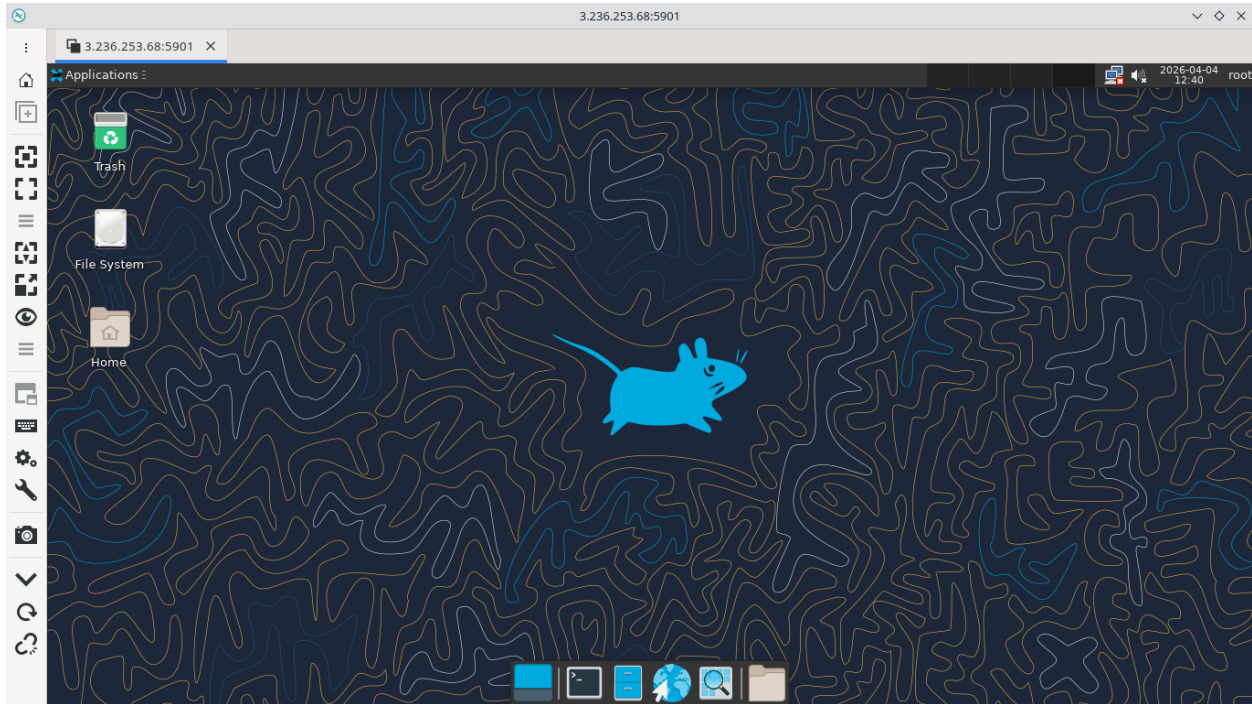
export XDG_SESSION_TYPE=x11
export XDG_CURRENT_DESKTOP=XFCE
export XDG_SESSION_DESKTOP=XFCE

exec startxfce4
EOF

chmod +x ~/.vnc/xstartup
echo -e "dog@123\ndog@123" | vncpasswd
exec vncserver -localhost no -geometry 1366x768 -fg
```

6.3 Usage

Use vnc client such as remmina connecting to `<ip_of_terminal_server>:5901` with password `dog@123`



WEB SERVER

7.1 Introduction

Apache HTTP Server, often called Apache httpd, is a widely used open-source web server that delivers web content to clients over the Internet using the HTTP and HTTPS protocols. It handles requests from browsers, serves static and dynamic content, supports modules for features like URL rewriting, authentication, and SSL/TLS encryption, and can host multiple websites on a single server. A web server, in general, is responsible for processing client requests, managing resources, and ensuring reliable delivery of web pages, applications, and services to users.

7.2 Implementation

```
#!/usr/bin/env bash

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

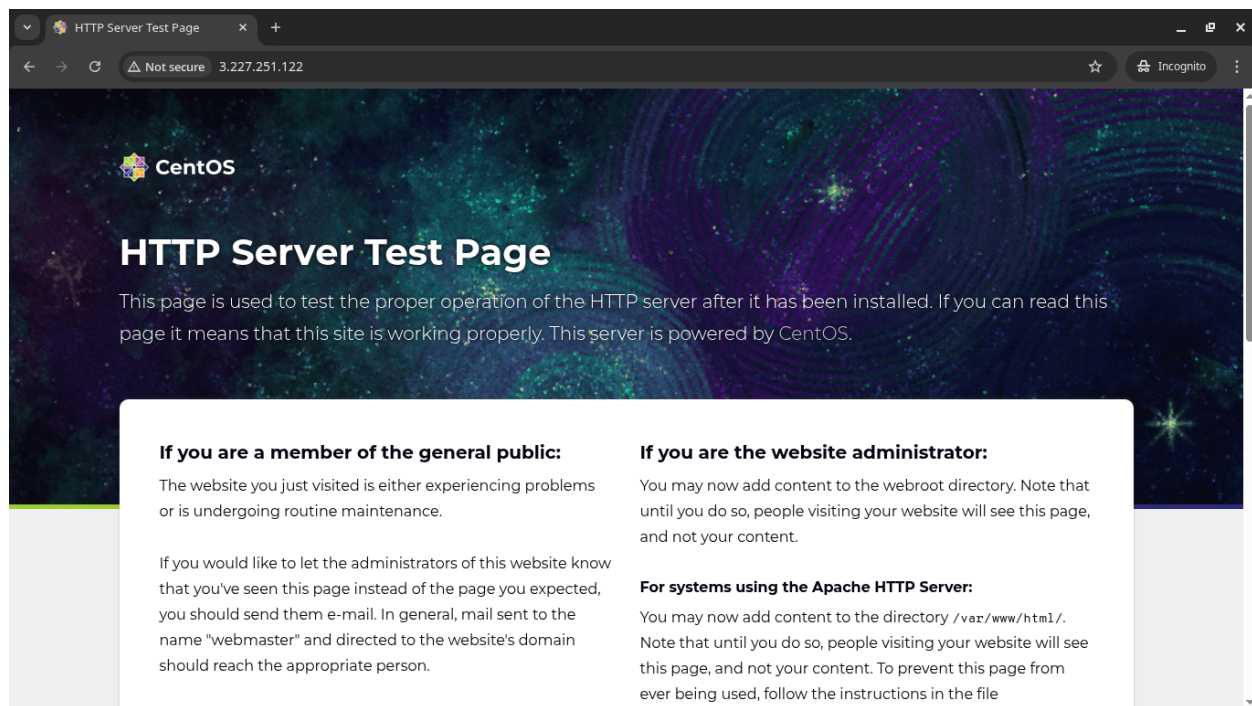
dnf install -y httpd

# # Create test page
# sudo tee /var/www/html/index.html > /dev/null <<EOF
# <!DOCTYPE html>
# <html>
# <head><title>CentOS Web Server</title></head>
# <body>
# <h1>Welcome to CentOS Stream 9 Web Server</h1>
# <p>Server is running successfully!</p>
# </body>
# </html>
# EOF

exec httpd -DFOREGROUND
```

7.3 Usage

- open firefox and visit `http://<ip_of_web_server>/`
- curl `http://<ip_of_web_server>/`



MAIL SERVER

8.1 Introduction

A mail server is a system that sends, receives, stores, and manages email messages over a network using standard protocols such as SMTP for sending and POP3 or IMAP for retrieving emails. It handles user authentication, message routing, spam filtering, and mailbox storage, enabling reliable communication between individuals and organizations. Mail servers can be configured for internal communication within a company or for public internet email services, and they often include security features like encryption and authentication to protect messages in transit and at rest.

8.2 Implementation

```
#!/usr/bin/env bash
set -e

echo "fastestmirror=True" >> /etc/dnf/dnf.conf
dnf install -y postfix dovecot cyrus-sasl cyrus-sasl-plain cyrus-sasl-lib
dnf clean all

# Dovecot
sed -i "s|ssl = required|ssl = no|" /
    ↳etc/dovecot/conf.d/10-ssl.conf
sed -i "s|ssl_cert = </etc/pki/dovecot/certs/dovecot.pem|ssl_cert = |" /
    ↳etc/dovecot/conf.d/10-ssl.conf
sed -i "s|ssl_key = </etc/pki/dovecot/private/dovecot.pem|ssl_key = |" /
    ↳etc/dovecot/conf.d/10-ssl.conf
sed -i "s|#disable_plaintext_auth = yes|disable_plaintext_auth = no|" /
    ↳etc/dovecot/conf.d/10-auth.conf
sed -i "s|auth_mechanisms = plain|auth_mechanisms = plain login|" /
    ↳etc/dovecot/conf.d/10-auth.conf

# Expose Dovecot SASL socket for Postfix
cat >> /etc/dovecot/conf.d/10-master.conf << 'EOF'

service auth {
    unix_listener /var/spool/postfix/private/auth {
        mode = 0660
        user = postfix
        group = postfix
    }
}
```

(continues on next page)

(continued from previous page)

```

EOF

# Postfix
postconf -e "myhostname = mail.mail.local"
postconf -e "mydomain = mail.local"
postconf -e "myorigin = \${mydomain}"

postconf -e "home_mailbox = Maildir/"
postconf -e "inet_interfaces = all"
postconf -e "inet_protocols = ipv4"

postconf -e "mydestination = \${myhostname}, localhost.\${mydomain}, localhost, \
↳\${mydomain}"
postconf -e "mynetworks = 127.0.0.0/8"

# SASL auth via Dovecot socket
postconf -e "smtpd_sasl_type = dovecot"
postconf -e "smtpd_sasl_path = private/auth"
postconf -e "smtpd_sasl_auth_enable = yes"
postconf -e "smtpd_sasl_security_options = noanonymous"
postconf -e "smtpd_sasl_local_domain = \${mydomain}"
postconf -e "broken_sasl_auth_clients = yes"

# Allow authenticated users to send
postconf -e "smtpd_relay_restrictions = permit_mynetworks, permit_sasl_authenticated, \
↳reject_unauth_destination"
postconf -e "smtpd_recipient_restrictions = permit_mynetworks, \
↳permit_sasl_authenticated, reject_unauth_destination"

newaliases

# Users
for user in testuser student; do
    id "$user" &>/dev/null || useradd -m "$user"
    echo "$user:password" | chpasswd
    mkdir -p /home/$user/Maildir/{cur,new,tmp}
    chown -R $user:$user /home/$user/Maildir
done

# Start
dovecot
postfix start

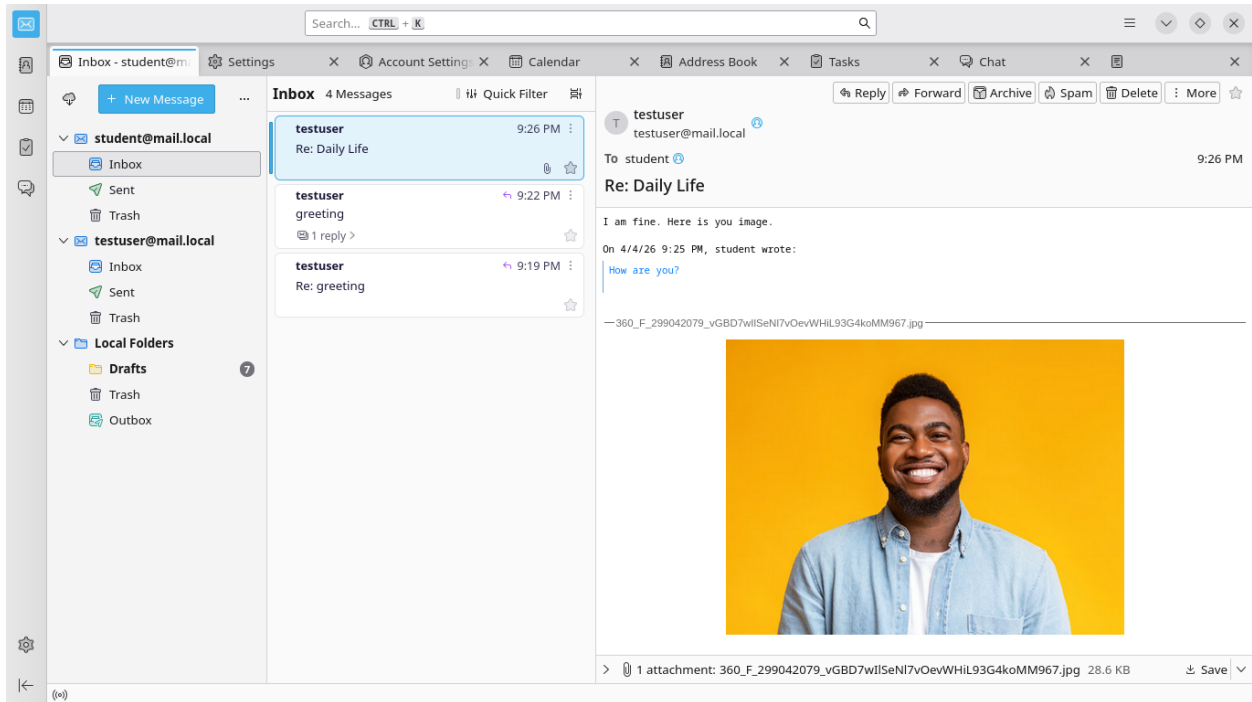
sleep infinity

```

8.3 Usage

- open thunderbird
- **email:** *student@mail.local*
 - username: student

- password: password
- email: **testuser@mail.local**
 - username: testuser
 - password: password
- IMAP - hostname: *<ip of mail server>* - port: 143
- SMTP - hostname: *<ip of mail server>* - port: 2525



DATABASE SERVER

9.1 Introduction

PostgreSQL, MongoDB, and Microsoft SQL Server are popular database management systems that store, organize, and manage data for applications. PostgreSQL and SQL Server are relational databases that use structured schemas and SQL queries for reliable transaction processing, while MongoDB is a NoSQL database optimized for flexible, document-oriented storage. A database server provides centralized data management, allowing multiple clients or applications to securely read, write, and manipulate data efficiently, ensuring consistency, availability, and scalability.

9.2 Implementation

```
#!/usr/bin/env bash

./postgresql.sh
./mongodb.sh
./sql_server.sh
# ./mysql.sh

exec sleep infinity
```

9.2.1 PostgreSQL

```
#!/usr/bin/env bash

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install -y postgresql-server postgresql-contrib

su -c "initdb -D /var/lib/pgsql/data" postgres
sed -i "s|#listen_addresses = 'localhost'|listen_addresses = '*'|" /
↪var/lib/pgsql/data/postgresql.conf
sed -i "s|#unix_socket_directories = '/var/run/postgresql, /tmp'
↪'|unix_socket_directories = ''|" /var/lib/pgsql/data/postgresql.conf
sed -i "s|host      all              all            127.0.0.1/32          trust|host      ↪
↪  all              all              0.0.0.0/0           trust|" /
↪var/lib/pgsql/data/pg_hba.conf

su -c "pg_ctl -D /var/lib/pgsql/data start" postgres
```

9.2.2 MongoDB

```
#!/usr/bin/env bash

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

# Add MongoDB repository
tee /etc/yum.repos.d/mongodb-org-7.0.repo > /dev/null <<EOF
[mongodb-org-7.0]
name=MongoDB Repository
baseurl=https://repo.mongodb.org/yum/redhat/9/mongodb-org/7.0/x86_64/
gpgcheck=1
enabled=1
gpgkey=https://pgp.mongodb.com/server-7.0.asc
EOF

# Install MongoDB
dnf install -y mongodb-org

sed -i "s|bindIp: 127.0.0.1|bindIp: 0.0.0.0|" /etc/mongod.conf

mongod -f /etc/mongod.conf --fork
```

9.2.3 SQL Server

```
#!/usr/bin/env bash

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

curl -o /etc/yum.repos.d/mssql-server.repo \
https://packages.microsoft.com/config/rhel/9/mssql-server-2022.repo
dnf install -y mssql-server

# curl -o /etc/yum.repos.d/msprod.repo \
# https://packages.microsoft.com/config/rhel/9/prod.repo
# dnf install -y mssql-tools18 unixODBC-devel

# Set environment variables to bypass interactive prompts if needed
export MSSQL_SA_PASSWORD='YourStrong!Password'
export ACCEPT_EULA='Y'

/opt/mssql/bin/mssql-conf setup <<< 1

/opt/mssql/bin/sqlservr > /dev/null 2>&1 &
```

9.3 Usage

9.3.1 PostgreSQL

```
psql -h <ip of postgresql> -p 5432 -U postgres

# +-----+
```

(continues on next page)

(continued from previous page)

```
# | Sample Output |
# +-----+
# jame@Jame-Linux ~/Desktop/coding/linux-server-assignment/docs % psql -h localhost -
→p 5432 -U postgres
# psql (18.2, server 13.23)
# Type "help" for help.
#
# postgres=#
```

```
/ # psql -h 44.210.244.117 -p 5432 -U postgres
psql (17.6, server 13.23)
Type "help" for help.

postgres=# CREATE TABLE cat(age int);
CREATE TABLE
postgres=# INSET INTO cat VALUES(11);
ERROR:  syntax error at or near "INSET"
LINE 1: INSET INTO cat VALUES(11);
        ^
postgres=# INSERT INTO cat VALUES(11);
INSERT 0 1
postgres=# INSERT INTO cat VALUES(18);
INSERT 0 1
postgres=# INSERT INTO cat VALUES(18);
postgres=# SELECT * FROM cat;
 age
-----
  11
  18
(2 rows)

postgres=#
```

9.3.2 MongoDB

```
mongosh <ip of mongodb>

# +-----+
# | Sample Output |
# +-----+
# root@Jame-Linux:/# mongosh
# Current MongoDB Log ID: 69a672534c3090e902d861df
# Connecting to:
→mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5
# Using MongoDB: 7.0.30
# Using Mongosh: 2.5.0
#
# For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
#
#
# To help improve our products, anonymous usage data is collected and sent to MongoDB.
→periodically (https://www.mongodb.com/legal/privacy-policy).
# You can opt-out by running the disableTelemetry() command.
#
# -----
```

(continues on next page)

(continued from previous page)

```
# The server generated these startup warnings when booting
# 2026-03-03T05:30:00.834+00:00: Access control is not enabled for the database.
↳ Read and write access to data and configuration is unrestricted
# 2026-03-03T05:30:00.835+00:00: You are running this process as the root user,
↳ which is not recommended
# 2026-03-03T05:30:00.835+00:00: Soft rlimits for open file descriptors too low
# 2026-03-03T05:30:00.835+00:00: For customers running MongoDB 7.0, we suggest
↳ changing the contents of the following sysfsFile
# -----
#
# test>
```

```
root@1bed15671a38:/# mongosh 44.210.244.117
Current Mongosh Log ID: 69d122bbafcbdd631bd861df
Connecting to:      mongodb://44.210.244.117:27017/?directConnection=true&appName=mongosh+2.5.0
Using MongoDB:      7.0.31
Using Mongosh:      2.5.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-04-04T13:00:28.550+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-04-04T13:00:28.550+00:00: You are running this process as the root user, which is not recommended
2026-04-04T13:00:28.551+00:00: vm.max_map_count is too low
-----
test> █
```

9.3.3 SQL Server

```
/opt/mssql-tools18/bin/sqlcmd -S <ip of sql server> -C -U sa -P 'YourStrong!Password'

# +-----+
# | Sample Output |
# +-----+
# [root@Jame-Linux /]# /opt/mssql-tools18/bin/sqlcmd -S localhost -C -U sa -P
↳ 'YourStrong!Password'
# 1> CREATE TABLE cat (name varchar(100), age int)
# 2> GO
# 1> INSERT INTO cat VALUES ('moo', 12)
# 2> GO
#
# (1 rows affected)
# 1> SELECT * FROM cat
# 2> GO
# name
↳
age
# -----
↳ -----
```

(continues on next page)

(continued from previous page)

```
# moo
↪ 12
#
# (1 rows affected)
# 1>
```

```
mssql@60f032a61991:/$ /opt/mssql-tools18/bin/sqlcmd -S 44.210.244.117 -C -U sa -P 'YourStrong!Password'
1> CREATE TABLE cow(id int)
2> GO
1> INSERT INTO cow VALUES(100)
2> GO

(1 rows affected)
1> INSERT INTO cow VALUES(193)
2> GO

(1 rows affected)
1> INSERT INTO cow VALUES(13)
2> GO

(1 rows affected)
1> SELECT * FROM cow
2> GO
id
-----
    100
    193
     13

(3 rows affected)
1> █
```


BACKUP SERVER

10.1 Introduction

Cronie is a time-based job scheduler for Linux systems that runs background tasks automatically at specified times using crontab configuration files, making it ideal for routine system maintenance like log rotation, updates, and automated backups. A backup server is a dedicated system designed to store copies of data from other machines to protect against data loss caused by hardware failure, human error, cyberattacks, or disasters; it centralizes backup management and enables recovery of files, databases, or entire systems when needed. Together, Cronie can automate scheduled backup tasks, while the backup server securely stores and manages the backup data.

10.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

# can use systemctl alternative for container
# curl -L
↪https://raw.githubusercontent.com/gdraheim/docker-systemctl-replacement/master/files/docker/systemctl
↪-o /usr/bin/systemctl && chmod +x /usr/bin/systemctl

dnf install -y cronie openssh-clients

mkdir /backup

cat <<"EOF" >> /etc/crontab
* * * * * root scp -P 2222 -o StrictHostKeyChecking=no -r_
↪root@FTP_SERVER_IP_PLACEHOLDER:/home/ftpuser /backup >> /log.txt && date >> /log.txt
EOF

mkdir -p /root/.ssh
cat <<"EOF" > /root/.ssh/id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnZAc1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAAMwAAAAAtzc2gtZW
QyNTUxOQAAACBcNrGfQQTSuDI1L1jyThNUqQpfzDGxyq/jcK1UDp/SpwAAAJgLVinuC1Yp
7gAAAAAtzc2gtZWQyNTUxOQAAACBcNrGfQQTSuDI1L1jyThNUqQpfzDGxyq/jcK1UDp/Spw
AAAEAC2bPdiHQtIbmXXYgGqQGhue7YITu6ANtAUD0Ygdacilw2sZ9BBNK4MiUvWPJOE1Sp
C1/MMbHKr+NwrVQOn9KnAAAAEXJvb3RAYWRjYtC4NTE0ZWRjAQIDBA==
-----END OPENSSH PRIVATE KEY-----
EOF
chmod 600 /root/.ssh/id_ed25519
```

(continues on next page)

(continued from previous page)

```
ssh-keyscan ftp_server >> /root/.ssh/known_hosts  
  
exec crond -n
```

10.3 Usage

The backup server will backup ftp server every one minute.

Connect to backup server instance with ssh.

```
docker exec -it linux-server-assignment-backup_server-1 bash  
ls /backup  
cat /log.txt
```

```
bash-5.1# ls /backup/ftpuser/ -a  
.  ..  .bash_logout  .bash_profile  .bashrc  
bash-5.1# tail /log.txt -f  
Sun Apr  5 06:05:01 UTC 2026  
Sun Apr  5 06:06:02 UTC 2026  
Sun Apr  5 06:07:01 UTC 2026  
Sun Apr  5 06:08:01 UTC 2026  
Sun Apr  5 06:09:01 UTC 2026  
Sun Apr  5 06:10:02 UTC 2026  
Sun Apr  5 06:11:01 UTC 2026  
Sun Apr  5 06:12:02 UTC 2026  
Sun Apr  5 06:13:01 UTC 2026  
Sun Apr  5 06:14:01 UTC 2026  
Sun Apr  5 06:15:02 UTC 2026  
^C  
bash-5.1# ls /backup/ftpuser/ -a  
.  ..  .bash_logout  .bash_profile  .bashrc  cow.txt  x.png  
bash-5.1# ls /backup/ftpuser/ -a  
.  ..  .bash_logout  .bash_profile  .bashrc  PostUser.bru  cow.txt  x.png  
bash-5.1# ls /backup/ftpuser/ -a  
.  ..  .bash_logout  .bash_profile  .bashrc  GetUser.bru  PostUser.bru  cow.txt  x.png  
bash-5.1#
```


LOAD BALANCING

11.1 Introduction

Load balancing is a technique used to distribute incoming network traffic or application requests across multiple servers to ensure no single server becomes overloaded. By spreading workloads evenly, it improves system performance, reliability, and availability, especially in high-traffic environments. Load balancers can operate at different layers of the network (such as transport or application layer) and may use methods like round-robin, least connections, or IP hashing to decide how requests are distributed.

11.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf install -y haproxy

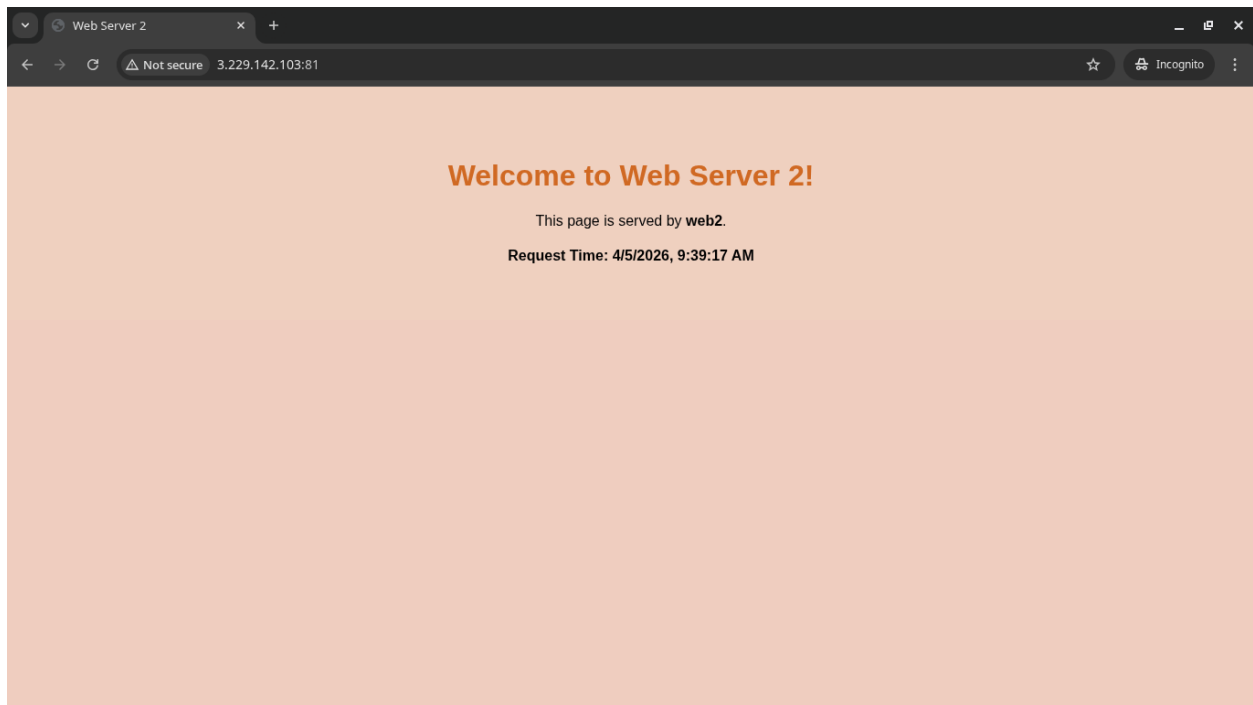
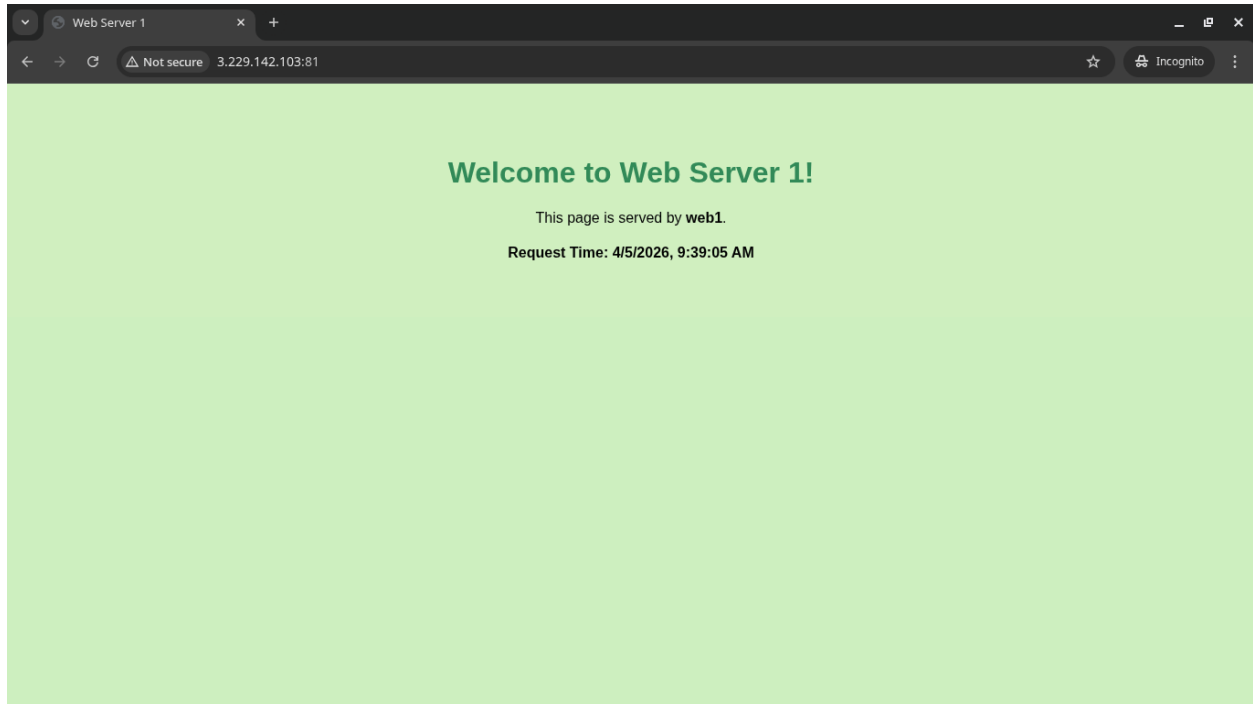
cat <<"EOF" >> /etc/haproxy/haproxy.cfg
frontend http_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    server web1 web1:80 check
    server web2 web2:80 check
EOF

exec haproxy -f /etc/haproxy/haproxy.cfg -db
```

11.3 Usage

vist *http://<ip of load balancing>:81*



FAILOVER CLUSTER

12.1 Introduction

PCS (Pacemaker/Corosync Configuration System) is a command-line tool used to configure and manage high-availability clusters; it provides an easier interface to control cluster components. **Pacemaker** is the cluster resource manager that decides where and how services (like web servers, databases, or virtual IPs) run, ensuring they fail over automatically if a node goes down. **Corosync** is the cluster communication engine that keeps nodes connected and synchronized by handling messaging, membership, and quorum. Together, PCS configures the cluster, Corosync manages node communication, and Pacemaker controls resource availability to provide reliable high-availability systems.

12.2 Implementation

```
#!/usr/bin/env bash

# +-----+
# | Run on each node |
# +-----+

echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf config-manager --set-enabled highavailability
dnf install -y pcs pacemaker corosync fence-agents-all iproute
dnf install -y nginx

echo "hacluster:password" | chpasswd
# passwd hacluster
systemctl start pcsd
```

```
#!/usr/bin/env bash

# +-----+
# | Run on one node only |
# +-----+

IP=$(ip -4 addr show eth0 | grep -oP 'inet \K[\d./]+' | cut -d/ -f1)

pcs host auth failover_cluster failover_cluster_2 -u hacluster

pcs cluster setup mycluster failover_cluster failover_cluster_2
pcs cluster start --all
```

(continues on next page)

(continued from previous page)

```

pcs property set stonith-enabled=false
pcs property set no-quorum-policy=ignore

pcs resource create vip ocf:heartbeat:IPaddr2 \
    ip="${IP%.*}.200" \
    cidr_netmask=16 \
    nic=eth0 \
    op monitor interval=30s

pcs resource create web \
    systemd:nginx \
    op monitor interval=30s

pcs resource group add web-stack vip web

```

12.3 Usage

Connect to failover cluster instance with ssh.

Use tmux to start multiple terminal sessions.

```
sudo dnf install -y tmux
```

12.3.1 Node1

```

docker exec -it linux-server-assignment-failover_cluster-1 bash
bash /root/failover_cluster.sh

# Make sure Node2 finish
bash /root/failover_cluster_one_node_only.sh # password `password`

pcs status
ip add

# +-----+
# | Sample Output |
# +-----+
# [root@cae754b673d6 /]# pcs status
# Cluster name: mycluster
# Cluster Summary:
#   * Stack: corosync (Pacemaker is running)
#   * Current DC: failover_cluster (version 2.1.10-2.el9-5693eae) - partition with
→quorum
#   * Last updated: Mon Mar  2 15:03:45 2026 on failover_cluster
#   * Last change:  Mon Mar  2 15:03:30 2026 by hacluster via hacluster on
→failover_cluster
#   * 2 nodes configured
#   * 2 resource instances configured
#
# Node List:

```

(continues on next page)

(continued from previous page)

```
# * Online: [ failover_cluster failover_cluster_2 ]
#
# Full List of Resources:
# * Resource Group: web-stack:
#   * vip (ocf:heartbeat:IPaddr2):          Started failover_cluster
#   * web (systemd:nginx):                  Started failover_cluster
#
# Daemon Status:
#   corosync: active/disabled
#   pacemaker: active/disabled
#   pcsd: active/disabled
# [root@cae754b673d6 /]# ip add
# 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default_
→qlen 1000
#   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
#   inet 127.0.0.1/8 scope host lo
#       valid_lft forever preferred_lft forever
#   inet6 ::1/128 scope host proto kernel_lo
#       valid_lft forever preferred_lft forever
# 2: eth0@if93: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP_
→group default
#   link/ether 62:a2:a0:e9:66:b1 brd ff:ff:ff:ff:ff:ff link-netnsid 0
#   inet 172.18.0.3/16 brd 172.18.255.255 scope global eth0
#       valid_lft forever preferred_lft forever
#   inet 172.18.0.200/16 brd 172.18.255.255 scope global secondary eth0
#       valid_lft forever preferred_lft forever
```

```
[root@e2e3c13665d7 /]# pcs status
Cluster name: mycluster
Cluster Summary:
  * Stack: corosync (Pacemaker is running)
  * Current DC: failover_cluster (version 2.1.10-2.el9-5693eae) - partition with quorum
  * Last updated: Sun Apr  5 06:55:13 2026 on failover_cluster
  * Last change:  Sun Apr  5 06:47:02 2026 by hacluster via hacluster on failover_cluster
  * 2 nodes configured
  * 2 resource instances configured

Node List:
  * Online: [ failover_cluster failover_cluster_2 ]

Full List of Resources:
  * Resource Group: web-stack:
    * vip          (ocf:heartbeat:IPaddr2):          Started failover_cluster
    * web          (systemd:nginx):                  Started failover_cluster

Daemon Status:
  corosync: active/disabled
  pacemaker: active/disabled
  pcsd: active/disabled
[root@e2e3c13665d7 /]#
```

[0] 0:docker* 1:docker- "e2e3c13665d7:/" 06:56 05-Apr-26

12.3.2 Node2

```
docker exec -it linux-server-assignment-failover_cluster_2-1 bash
bash /root/failover_cluster.sh
```

FTP SERVER

13.1 Introduction

An FTP server is a system that uses the File Transfer Protocol (FTP) to allow users to upload, download, and manage files over a network. It enables file sharing between clients and a central server using authentication with usernames and passwords, and can support both anonymous and secure access configurations. FTP servers are commonly used for website file management, internal file distribution, and data exchange between systems, and can be secured using extensions like FTPS to encrypt data during transmission.

13.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

# install sshd for backup
dnf install -y openssh-server
mkdir -p /root/.ssh
cat <<"EOF" > /root/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFw2sZ9BBNK4MiUvWPJ0E1SpCl/MMbHKr+NwrVQOn9Kn...
↪root@adca78514edc
EOF
chmod 600 /root/.ssh/authorized_keys
ssh-keygen -A
/usr/sbin/sshd

dnf install -y --allowdowngrading vsftpd curl
dnf clean all

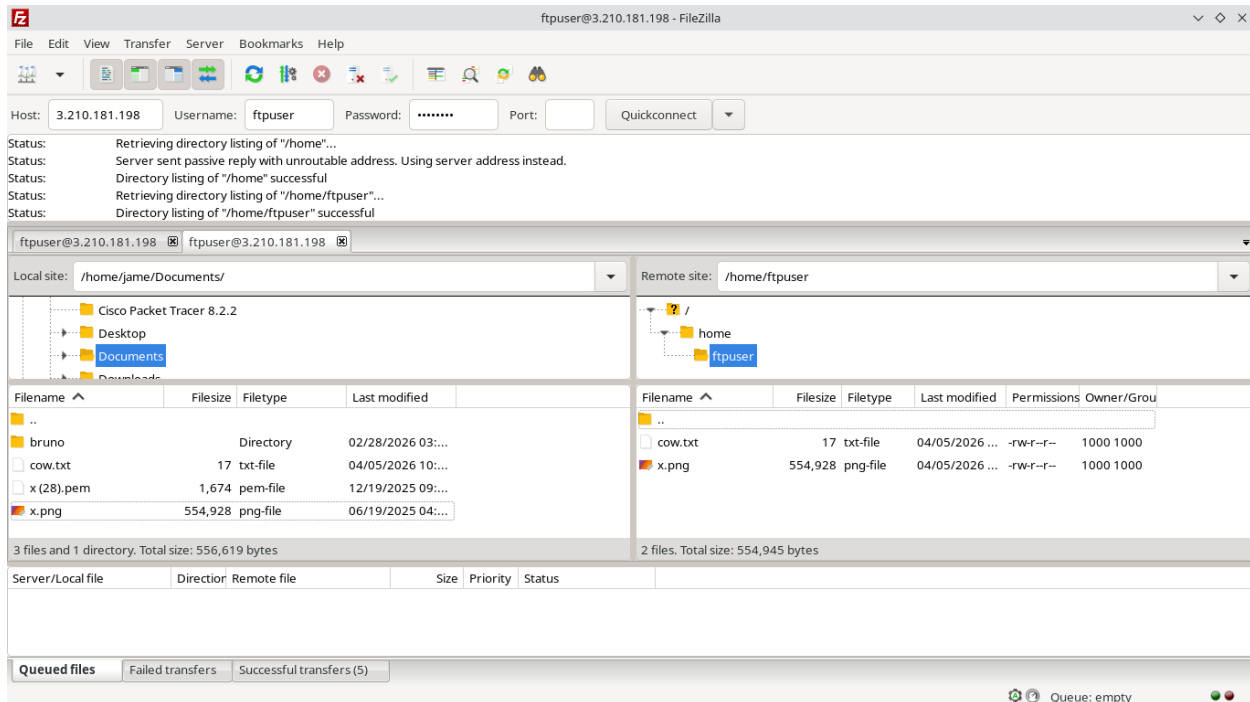
PUBLIC_IP=$(curl http://checkip.amazonaws.com)
useradd -m ftpuser && echo "ftpuser:password" | chpasswd

# Configure vsftpd for Passive Mode
echo "pasv_enable=YES" >> /etc/vsftpd/vsftpd.conf
echo "pasv_min_port=21100" >> /etc/vsftpd/vsftpd.conf
echo "pasv_max_port=21110" >> /etc/vsftpd/vsftpd.conf
echo "pasv_address=${PUBLIC_IP}" >> /etc/vsftpd/vsftpd.conf

vsftpd
sleep infinity
```

13.3 Usage

- use filezilla
- host: <ip of ftp server>
- port: 21
- username: ftpuser
- password: password



DOCKER

14.1 Introduction

Docker is an open-source containerization platform that allows developers to package applications and their dependencies into lightweight, portable containers that run consistently across different environments. Instead of relying on full virtual machines, Docker uses operating system-level virtualization to isolate applications while sharing the host kernel, making it faster and more resource-efficient. It simplifies development, testing, and deployment workflows by ensuring that applications behave the same way on a developer's laptop, a server, or in the cloud.

14.2 Implementation

```
#!/usr/bin/env bash
echo "fastestmirror=True" >> /etc/dnf/dnf.conf

dnf -y install dnf-plugins-core
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
dnf -y install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
↪ docker-compose-plugin

# curl -L
↪ https://raw.githubusercontent.com/gdraheim/docker-systemctl-replacement/master/files/docker/systemd
↪ -o /usr/bin/systemctl && chmod +x /usr/bin/systemctl

cat <<"EOF" > /etc/docker/daemon.json
{
  "storage-driver": "vfs"
}
EOF

# systemctl start docker
dockerd &

# install sshd for remote ssh
dnf install -y openssh-server
sed -i "s|#PermitRootLogin prohibit-password|PermitRootLogin yes|" /
↪ etc/ssh/sshd_config
echo "root:password" | chpasswd
ssh-keygen -A
/usr/sbin/sshd
```

(continues on next page)

(continued from previous page)

```
sleep infinity
```

14.3 Usage

ssh to *root@<ip of docker instance>* with port 8081 and password 'password'

```
jame@Jame-Linux:~/Desktop/coding/linux-server-assignment|
➔ ssh root@3.210.181.198 -p 8081
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@3.210.181.198's password:
Last login: Sun Apr  5 04:14:53 2026 from 58.97.216.163
[root@1b3dff3a1098 ~]# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
[root@1b3dff3a1098 ~]# docker run -it ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
root@7ac41c62e25f:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
```

DOMAIN CONTROLLER

15.1 Introduction

FreeIPA is a free, open-source Identity Management (IdM) solution designed primarily for Linux and Unix-based networked environments. Often described as the Linux equivalent of Microsoft Active Directory, it provides a centralized platform for managing authentication, authorization, and network policies.

15.2 Implementation

```
#!/usr/bin/env bash

# installing freeipa with docker
# docker run -h ipa.example.test --read-only --cgroupns=host -v /
↪ sys/fs/cgroup:/sys/fs/cgroup:rw -ti \
# -e PASSWORD=Secret123 \
# freeipa/freeipa-server:centos-9-stream ipa-server-install -U -r
↪ EXAMPLE.TEST --no-ntp --skip-mem-check

# manually installing freeipa
# echo "fastestmirror=True" >> /etc/dnf/dnf.conf
# dnf -y install freeipa-server freeipa-server-dns
# ipa-server-install --setup-dns

# for client
# docker run -it --privileged --cgroupns=host --tmpfs=/run --tmpfs=/tmp --
↪ volume=/sys/fs/cgroup:/sys/fs/cgroup:rw trfore/docker-centos9-systemd:latest
```

15.3 Usage

Connect to domain controller instance with ssh.

15.3.1 Domain Controller

```
# make sure the freeipa server is running completely
docker exec -it linux-server-assignment-domain_controller-1 bash

kinit admin # password `Secret123`
ipa user-add testuser --first=Test --last=User --password
```

```
[root@ipa ~]# kinit admin # password `Secret123`
Password for admin@EXAMPLE.TEST:
[root@ipa ~]# ipa user-add testuser --first=Test --last=User --password
Password:
Enter Password again to verify:
-----
Added user "testuser"
-----
  User login: testuser
  First name: Test
  Last name: User
  Full name: Test User
  Display name: Test User
  Initials: TU
  Home directory: /home/testuser
  GECOS: Test User
  Login shell: /bin/sh
  Principal name: testuser@EXAMPLE.TEST
  Principal alias: testuser@EXAMPLE.TEST
  User password expiration: 20260405091315Z
  Email address: testuser@example.test
  UID: 1287000004
  GID: 1287000004
  Password: True
  Member of groups: ipausers
  Kerberos keys available: True
[root@ipa ~]#
```

15.3.2 Client

```
# make sure to run this after the domain controller usage
docker exec -it linux-server-assignment-centos9_systemd-1 bash

dnf -y install freeipa-client
echo "client1.example.test" > /etc/hostname
hostname client1.example.test
ipa-client-install --mkhomedir

id admin
getent passwd admin
id testuser
getent passwd testuser

su admin
su testuser
```

```
[root@client1 /]# ipa-client-install --mkhomedir
This program will set up IPA client.
Version 4.13.1

DNS discovery failed to determine your DNS domain
Provide the domain name of your IPA server (ex: example.com): example.test
Provide your IPA server name (ex: ipa.example.com): ipa.example.test
The failure to use DNS to find your IPA server indicates that your resolv.conf file is not properly configured.
Autodiscovery of servers for failover cannot work with this configuration.
If you proceed with the installation, services will be configured to always access the discovered server for all operations
and will not fail over to other servers in case of failure.
Proceed with fixed values and no DNS discovery? [no]: yes
Do you want to configure chrony with NTP server or pool address? [no]:
Client hostname: client1.example.test
Realm: EXAMPLE.TEST
DNS Domain: example.test
IPA Server: ipa.example.test
BaseDN: dc=example,dc=test

Continue to configure the system with these values? [no]: yes
Synchronizing time
No SRV records of NTP servers found and no NTP server or pool address was provided.
Using default chrony configuration.
Attempting to sync time with chronyc.
Time synchronization was successful.
User authorized to enroll computers: admin
Password for admin@EXAMPLE.TEST:
Successfully retrieved CA cert
    Subject:      CN=Certificate Authority,O=EXAMPLE.TEST
```

```
Successfully retrieved CA cert
    Subject:      CN=Certificate Authority,O=EXAMPLE.TEST
    Issuer:       CN=Certificate Authority,O=EXAMPLE.TEST
    Valid From:   2026-04-05 08:24:01+00:00
    Valid Until:  2026-04-05 08:24:01+00:00

Enrolled in IPA realm EXAMPLE.TEST
Created /etc/ipa/default.conf
Configured /etc/sss/sss.conf
Systemwide CA database updated.
Hostname (client1.example.test) does not have A/AAAA record.

Failed to update DNS records.
Missing A/AAAA record(s) for host client1.example.test: 172.18.0.2.
Incorrect reverse record(s):
172.18.0.2 is pointing to linux-server-assignment-centos9_systemd-1.linux-server-assignment_default. instead of client1.example.test.
SSSD enabled
Configured /etc/openldap/ldap.conf
/etc/ssh/ssh_config not found, skipping configuration
/etc/ssh/sshd_config not found, skipping configuration
Configuring example.test as NIS domain.
Configured /etc/krb5.conf for IPA realm EXAMPLE.TEST
Client configuration complete.
The ipa-client-install command was successful
[root@client1 /]#
[root@client1 /]# id admin
uid=1287000000(admin) gid=1287000000(admins) groups=1287000000(admins)
[root@client1 /]# getent passwd admin
```

```
uid=1287000000(admin) gid=1287000000(admins) groups=1287000000(admins)
[root@client1 /]# getent passwd admin
admin:*:1287000000:1287000000:Administrator:/home/admin:/bin/bash
[root@client1 /]# id testuser
uid=1287000003(testuser) gid=1287000003(testuser) groups=1287000003(testuser)
[root@client1 /]# getent passwd testuser
testuser:*:1287000003:1287000003:Test User:/home/testuser:/bin/sh
[root@client1 /]# su admin
bash-5.1$ exit
exit
[root@client1 /]# su admin
bash-5.1$ whoami
admin
bash-5.1$ exit
exit
[root@client1 /]# su testuser
sh-5.1$ whoami
testuser
sh-5.1$ su admin
Password:
bash-5.1$ whoami
admin
bash-5.1$ exit
exit
sh-5.1$ exit
exit
[root@client1 /]# whoami
root
[root@client1 /]#
```

COMPOSE YAML

The `compose.yaml` file defines an ambitious, all-in-one infrastructure lab that effectively functions as a “data center in a box.” Utilizing CentOS Stream 9 as its primary backbone, the configuration orchestrates a massive array of services ranging from standard web and database servers (supporting PostgreSQL, MongoDB, and others) to critical networking components like VPNs, DNS, and DHCP. It even ventures into high-availability territory with failover clusters and load balancing, while relying on locally mounted shell scripts to automate the configuration of each container. It is a highly comprehensive, modular playground designed for simulating a complex enterprise environment.

```
# EXPOSE/expose => no-op/documentation (modern) [can be used with -P]
#               => ... (old)

services:
  file_server:
    image: quay.io/centos/centos:stream9
    volumes:
      - ./src/file_server/file_server.sh:/root/file_server.sh:ro
    working_dir: /root
    command: bash /root/file_server.sh
    ports:
      - "445:445"
  proxy_server:
    image: quay.io/centos/centos:stream9
    volumes:
      - ./src/proxy_server/proxy_server.sh:/root/proxy_server.sh:ro
    working_dir: /root
    command: bash /root/proxy_server.sh
    ports:
      - "3128:3128"
  dns_server:
    image: quay.io/centos/centos:stream9
    volumes:
      - ./src/dns_server/dns_server.sh:/root/dns_server.sh:ro
    working_dir: /root
    command: bash /root/dns_server.sh
    ports:
      - "0.0.0.0:53:53"
      - "0.0.0.0:53:53/udp"
  terminal_server:
    image: quay.io/centos/centos:stream9
    volumes:
      - ./src/terminal_server/terminal_server.sh:/root/terminal_server.sh:ro
    working_dir: /root
```

(continues on next page)

(continued from previous page)

```

command: bash /root/terminal_server.sh
ports:
  - "0.0.0.0:5901:5901"
web_server:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/web_server/web_server.sh:/root/web_server.sh:ro
  working_dir: /root
  command: bash /root/web_server.sh
  ports:
    - "0.0.0.0:80:80"
database_server:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/database_server/database_server.sh:/root/database_server.sh:ro
    - ./src/database_server/postgresql.sh:/root/postgresql.sh:ro
    - ./src/database_server/mongodb.sh:/root/mongodb.sh:ro
    - ./src/database_server/sql_server.sh:/root/sql_server.sh:ro
    - ./src/database_server/mysql.sh:/root/mysql.sh:ro
  working_dir: /root
  command: bash /root/database_server.sh
  ports:
    - "0.0.0.0:5432:5432" # postgresql
    - "0.0.0.0:27017:27017" # mongodb
    - "0.0.0.0:1433:1433" # sql server
    - "0.0.0.0:3306:3306" # mysql
mail_server:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/mail_server/mail_server.sh:/root/mail_server.sh:ro
  working_dir: /root
  command: bash /root/mail_server.sh
  ports:
    - "0.0.0.0:2525:25" # Because of widespread abuse by spammers and malware, most
    ↪ISP and cloud providers block port 25 for end-user submissions (e.g., using
    ↪Outlook/Thunderbird)
    - "0.0.0.0:143:143"
    - "0.0.0.0:110:110"
ftp_server:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/ftp_server/ftp_server.sh:/root/ftp_server.sh:ro
  working_dir: /root
  command: bash /root/ftp_server.sh
  ports:
    - "0.0.0.0:21:21"
    - "0.0.0.0:21100-21110:21100-21110" # Expose Passive FTP ports
    - "0.0.0.0:2222:22" # ADD THIS to expose SSH for scp
load_balancing:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/load_balancing/load_balancing.sh:/root/load_balancing.sh:ro

```

(continues on next page)

(continued from previous page)

```

working_dir: /root
command: bash /root/load_balancing.sh
depends_on:
  - web1
  - web2
ports:
  - "0.0.0.0:81:80"
web1:
  image: nginx:stable-alpine
  volumes:
    - ./src/load_balancing/web1.html:/usr/share/nginx/html/index.html:ro
web2:
  image: nginx:stable-alpine
  volumes:
    - ./src/load_balancing/web2.html:/usr/share/nginx/html/index.html:ro
backup_server:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/backup_server/backup_server.sh:/root/backup_server.sh:ro
  working_dir: /root
  command: bash /root/backup_server.sh
  depends_on:
    - ftp_server
docker:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/docker/docker.sh:/root/docker.sh:ro
  working_dir: /root
  command: bash /root/docker.sh
  privileged: true
  ports:
    - "0.0.0.0:8081:22"
domain_controller:
  image: freeipa/freeipa-server:centos-9-stream
  hostname: ipa.example.test
  read_only: true
  cgroup: host
  stdin_open: true
  tty: true
  environment:
    PASSWORD: Secret123
  volumes:
    - /sys/fs/cgroup:/sys/fs/cgroup:rw
    - ./src/domain_controller/domain_controller.sh:/root/domain_controller.sh:ro
  working_dir: /root
  command: >
    ipa-server-install -U -r EXAMPLE.TEST --no-ntp --skip-mem-check
  depends_on:
    - centos9_systemd
centos9_systemd:
  image: trfore/docker-centos9-systemd:latest
  privileged: true

```

(continues on next page)

(continued from previous page)

```

cgroup: host
stdin_open: true
tty: true
tmpfs:
  - /run
  - /tmp
volumes:
  - /sys/fs/cgroup:/sys/fs/cgroup:rw
vpn_server:
  image: quay.io/centos/centos:stream9
  cap_add:
    - NET_ADMIN          # Required for tun device
  devices:
    - /dev/net/tun       # TUN device
  volumes:
    - ./src/vpn_server/vpn_server.sh:/root/vpn_server.sh:ro
    - openvpn-data:/etc/openvpn
  working_dir: /root
  command: bash /root/vpn_server.sh
  depends_on:
    - vpn_client
  ports:
    - "1194:1194/udp"
vpn_client:
  image: quay.io/centos/centos:stream9
  cap_add:
    - NET_ADMIN          # Required for tun device
  devices:
    - /dev/net/tun       # TUN device
  volumes:
    - ./src/vpn_server/vpn_client.sh:/root/vpn_client.sh:ro
    - openvpn-data:/etc/openvpn
  working_dir: /root
  command: sleep infinity
dhcp_server:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/dhcp_server/dhcp_server.sh:/root/dhcp_server.sh:ro
  working_dir: /root
  command: bash /root/dhcp_server.sh
  depends_on:
    - dhcp_client
  # ports:                # reserve for production use
  # - "67:67/udp" # if dhcp server listen on 127.0.0.1:67, it is impossible for
↪any client (including in the same machine) to connect to the dhcp server
  # - "68:68/udp"
dhcp_client:
  image: quay.io/centos/centos:stream9
  volumes:
    - ./src/dhcp_server/dhcp_client.sh:/root/dhcp_client.sh:ro
  working_dir: /root
  command: sleep infinity

```

(continues on next page)

(continued from previous page)

```
failover_cluster:
  image: trfore/docker-centos9-systemd:latest
  privileged: true
  cgroup: host
  stdin_open: true
  tty: true
  tmpfs:
    - /run
    - /tmp
  volumes:
    - /sys/fs/cgroup:/sys/fs/cgroup:rw
    - ./src/failover_cluster/failover_cluster.sh:/root/failover_cluster.sh:ro
    - ./src/failover_cluster/failover_cluster_one_node_only.sh:/root/failover_cluster_one_node_only.sh:ro
  depends_on:
    - failover_cluster_2
failover_cluster_2:
  image: trfore/docker-centos9-systemd:latest
  privileged: true
  cgroup: host
  stdin_open: true
  tty: true
  tmpfs:
    - /run
    - /tmp
  volumes:
    - /sys/fs/cgroup:/sys/fs/cgroup:rw
    - ./src/failover_cluster/failover_cluster.sh:/root/failover_cluster.sh:ro
volumes:
  openvpn-data:
```